

Relatório Técnico — Decibéis: a turnê contra O Empresário

Disciplina: Programação Orientada a Objetos — UTFPR · 2026 **Autor:** João Prissão (JoaoPrissao)

Repositório: <https://github.com/JoaoPrissao/Projeto-POO> (branch `main`) **Data:** Junho de 2026

Sumário

1. Introdução
2. Processo de Desenvolvimento
3. Atendimento aos Requisitos da Disciplina
4. Arquitetura
5. Padrões de Projeto
6. Herança e Polimorfismo
7. Sobrecarga de Operadores
8. Qualidade e Testes
9. Decisões de Design
10. Limitações Conhecidas
11. Conclusão

1. Introdução

1.1 Objetivo do Projeto

Este projeto é o trabalho final da disciplina de Programação Orientada a Objetos. A proposta foi construir um jogo de RPG completo e jogável que demonstrasse, em código executável, os principais conceitos da POO: herança, polimorfismo, encapsulamento, abstração, padrões de projeto GoF e sobrecarga de operadores.

Optei por desenvolver em **Python 3.12**, com a interface gráfica rodando em **pywebview**. Essa escolha me permitiu manter toda a lógica de jogo em Python puro (a parte que de fato exercita os conceitos de POO avaliados) e usar HTML/CSS/JS apenas para a camada de apresentação. A disciplina ("POO com Python e C++") aceita Python como linguagem de entrega; onde a especificação cita construções de C++, a implementação usa o equivalente direto da linguagem — classes abstratas via `ABC` / `@abstractmethod` no lugar de funções virtuais puras, módulos no lugar de namespaces, e `pytest` no lugar de `gtest`.

1.2 Tema Escolhido: Banda de Rock em Turnê

Em vez do RPG de fantasia clássico (guerreiros e magos), escolhi o tema de uma **banda de rock em turnê**. Quatro músicos — Guitarrista, Vocalista, Baterista e Baixista — percorrem venues (bar, feira, arena) enfrentando capangas até chegar ao chefe final, O Empresário. A troca de tema não muda os conceitos de

POO exercitados (continua sendo uma hierarquia de personagens com atributos e ataques distintos), mas tornou o projeto mais autoral e divertido de testar.

O resultado é um jogo jogável de ponta a ponta, com três sistemas principais:

- **Overworld 2D:** a van da banda percorre um mapa em side-scroll; o jogador para em venues, conversa com NPCs e abre baús.
- **Batalha por turnos:** cada músico ataca com 3 golpes (leve/médio/pesado); o vilão revida; um sistema de energia e cansaço dá profundidade estratégica.
- **Minigame de ritmo:** antes de cada ataque o jogador pressiona teclas no ritmo certo para multiplicar o dano — acertos perfeitos formam combos.

2. Processo de Desenvolvimento

Desenvolvi o projeto de forma incremental, fechando uma camada de cada vez e só avançando quando a anterior estava testada. A ordem abaixo reflete como o código foi de fato crescendo:

1. **Modelagem do domínio.** Comecei pela classe abstrata `Musico` e suas quatro subclasses, definindo HP, XP, energia e level-up na base. Em paralelo modelei `Item` (com `Equipavel` e `Consumivel`) e o `Inventario`. Essa primeira camada não dependia de interface nenhuma — eu a testava direto pelo terminal.
 2. **Sistema de combate.** Com os músicos prontos, escrevi a classe `Show`, que orchestra o turno: ação do músico, dano, resposta do inimigo, energia e cansaço. O chefe `Empresario` entrou aqui como um inimigo com comportamento próprio.
 3. **Tratamento de erros.** Conforme o domínio crescia, percebi que precisava de erros tipados em vez de `ValueError` genérico. Montei a hierarquia de exceções a partir de `JogoError`, o que me deixou capturar falhas no nível certo (só de inventário, só de campanha, etc.).
 4. **Campanha e progressão.** Implementei a `Campanha` para amarrar as venues: fama acumulada, cachê, cooldowns e o desbloqueio da Arena. Foi o que transformou batalhas soltas em uma progressão com começo, meio e fim.
 5. **Interface gráfica.** Só depois de a lógica estar sólida liguei o frontend. Criei a camada `bridge` (pywebview) para traduzir as chamadas do JavaScript para o domínio Python, mantendo a regra de que o backend é a autoridade do estado e o frontend apenas renderiza.
 6. **Polimento jogável.** Com o jogo rodando, adicionei o minigame de ritmo, o áudio por venue, a loja, opções de volume e o modo tela cheia (F11). Vários ajustes nesta etapa vieram de testar o jogo na prática e perceber o que estava confuso ou desbalanceado.
 7. **Persistência.** Adicionei save/load em JSON por slot, para o jogador retomar a turnê.
 8. **Testes e documentação.** Em paralelo a tudo acima, mantive uma suíte de testes com `pytest`. Alguns recursos (como os dunder do inventário) nasceram por TDD: escrevi o teste primeiro e só depois a implementação. Por fim, produzi o diagrama UML, este relatório e as instruções de empacotamento.
-

3. Atendimento aos Requisitos da Disciplina

A especificação pede dois requisitos funcionais centrais: **criação de personagem com escolhas** e **level-up com pontos de experiência**. Adaptei os dois ao tema da banda sem abrir mão da mecânica exigida.

3.1 Criação de Personagem com Escolhas

Em vez de um único protagonista, o jogador monta uma **banda de 4 músicos**, escolhendo nome e tipo para cada membro. Cada tipo tem atributos iniciais distintos e uma fórmula de ataque própria:

Tipo	Atributo-chave	Valor inicial	Mecânica de ataque
Guitarrista	força	10	<code>força * 1.5 + ego</code>
Vocalista	inteligencia	10	<code>inteligencia * 1.8</code>
Baterista	agilidade	10	<code>agilidade * 1.2 + chance de crítico</code>
Baixista	<code>força (via Guitarrista) + fe</code>	—	<code>força * 1.3 + fe * 0.5</code>

A criação passa por `MusicoFactory.criar(tipo, nome=...)` em `backend/fabricas.py`, chamada por `criar_banda(composicao)` em `bridge/api.py`, que recebe a lista de `{tipo, nome}` preenchida na tela `#tela-criar-banda`. A "escolha de personagem" do requisito vira, no jogo, a composição da banda.

3.2 Level-up com Pontos de XP

O XP representa a **experiência de turnê** — shows realizados com sucesso — e subir de nível significa a reputação crescente do músico, com um bônus de +10 de HP por nível representando o preparo físico de quem toca ao vivo:

```
def ganhar_xp(self, quantidade: int) -> None:
    if quantidade <= 0:
        return
    self._xp += quantidade
    while self._xp >= self._xp_proximo_nivel:
        self._xp -= self._xp_proximo_nivel
        self.subir_nivel()

def subir_nivel(self) -> None:
    self._nivel += 1
    self._xp_proximo_nivel = self._nivel * self.XP_BASE # 100, 200, 300...
    bonus_hp = 10
    self._hp_maximo += bonus_hp
    self._hp = min(self._hp + bonus_hp, self._hp_maximo)
```

O XP é concedido ao concluir uma venue (`concluir_venue()` em `bridge/api.py`). O level-up é automático ao atingir o limiar — sem ponto de gasto manual, porque a progressão é orgânica à turnê. O excedente além do limiar é preservado e carregado para o próximo nível, o que mantém a coerência quando o músico sobe vários níveis de uma vez.

Requisito	Implementação no jogo	Arquivo
Criação de personagem com escolhas	Composição da banda (4 músicos, tipos distintos)	<code>fabricas.py</code> , <code>api.py</code>
Level-up com XP	Experiência de turnê → reputação → +HP	<code>musico.py</code>

4. Arquitetura

4.1 Visão Geral — Três Camadas

O projeto segue uma arquitetura em **três camadas** com separação estrita de responsabilidades:

Frontend (HTML/CSS/JS) frontend/index.html + js/{main,batalha,...}.js Só renderiza; nunca decide regra de jogo
Bridge (pywebview) bridge/api.py → @_ponte (ErroDTO) Traduz chamadas JS ↔ domínio Python
Backend (Python puro) backend/{musico,show,campanha,gerenciador,...} Autoridade absoluta do estado de jogo

Backend — lógica de domínio em Python OOP puro. Sem I/O de UI; comunica erros via exceções. **Bridge** — @_ponte captura qualquer exceção do domínio e converte para um DTO JSON {ok, erro}. **Frontend** — renderiza o estado recebido do backend; não valida regras nem mantém estado canônico.

Mantive essa separação desde o começo justamente para conseguir testar todo o domínio sem precisar abrir a interface — os 361 testes rodam sem subir o pywebview.

4.2 Diagrama UML de Classes

O diagrama completo está em docs/uml_classes.md (e renderizado em uml_classes.png / uml_classes.svg) — um diagrama de classes coerente com a implementação real, incluindo as três hierarquias (Musico, Item, JogoError) e as relações de Singleton, Factory e Bridge.

5. Padrões de Projeto

Identifiquei **6 padrões GoF** no código. Os 4 primeiros são os canônicos exigidos pela rubrica; os 2 últimos surgiram naturalmente da arquitetura e entram como bônus.

5.1 Singleton — GerenciadorJogo

Categoria GoF: Criacional · **Arquivo:** backend/gerenciador.py

Garante que existe **uma única instância** do estado central do jogo em toda a execução.

```
class GerenciadorJogo:
    _instancia: "GerenciadorJogo | None" = None

    def __new__(cls) -> "GerenciadorJogo":
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
            cls._instancia._inicializado = False
        return cls._instancia
```

Qualquer módulo que instancie `GerenciadorJogo()` recebe o mesmo objeto. Testado em `tests/test_singleton.py`.

5.2 Factory Method — MusicoFactory e ItemFactory

Categoria GoF: Criacional · **Arquivo:** `backend/fabricas.py`

Centraliza a criação de músicos e itens a partir de uma string de tipo. O dicionário `_tipos` é o **único ponto que cita classes concretas** — novos tipos entram via `registrar()` sem alterar o código existente (princípio Open/Closed).

```
class MusicoFactory:
    _tipos: dict[str, type[Musico]] = {
        "guitarrista": Guitarrista,
        "vocalista": Vocalista,
        "baterista": Baterista,
        "baixista": Baixista,
    }

    @classmethod
    def criar(cls, tipo: str, **kwargs) -> Musico:
        if tipo not in cls._tipos:
            raise TipoInvalidoError(tipo)
        return cls._tipos[tipo](**kwargs)

    @classmethod
    def registrar(cls, chave: str, classe: type[Musico]) -> None:
        cls._tipos[chave] = classe
```

`ItemFactory` segue a mesma estrutura para itens (`Equipavel`, `Consumivel`). Testado em `tests/test_fabricas.py`.

5.3 Template Method — Musico (ABC) e Subclasses

Categoria GoF: Comportamental · **Arquivo:** `backend/musico.py` + subclasses

`Musico` é uma classe abstrata (ABC) que define o **esqueleto do algoritmo** de personagem: HP, XP, energia, level-up e equipamento, todos implementados na base. O único **passo variável** é `atacar()`, obrigatoriamente sobrescrito por cada subclasse.

```
class Musico(ABC):
    @abstractmethod
    def atacar(self) -> int:
        """Retorna o dano base sem aplicá-lo."""
        pass

    def subir_nivel(self) -> None: # algoritmo fixo herdado por todos
        self._nivel += 1
        self._xp_proximo_nivel = self._nivel * self.XP_BASE
        ...
```

Cada subclasse implementa `atacar()` conforme seu atributo (força, inteligência, agilidade): a estrutura é fixa na base e o passo concreto fica nas folhas.

5.4 Strategy — Moveset (moves.py)

Categoria GoF: Comportamental · **Arquivo:** `backend/moves.py` + `backend/show.py`

O jogador escolhe **qual golpe usar em cada turno** (leve, médio, pesado, especial de equipamento). Cada golpe encapsula a estratégia de ataque — multiplicador de dano, custo de energia, efeito de cansaço:

```
MOVES_BASE = {
    "guitarrista": [
        {"id": "leve", "mult": 0.8, "custo": 10, "cansa": False},
        {"id": "medio", "mult": 1.0, "custo": 25, "cansa": False},
        {"id": "pesado", "mult": 1.5, "custo": 40, "cansa": True},
    ],
    ...
}
```

`Show.acao_musico()` recebe o move e aplica `mult_extra`, `custo_energia` e `cansa` sem saber qual move foi escolhido — o algoritmo de ataque é selecionado em runtime.

5.5 Hierarquia de Exceções — `JogoError`

Categoria GoF: Estrutural · **Arquivo:** `backend/excecoes.py`

Toda exceção do jogo deriva de `JogoError`, formando uma **árvore tipada de 26 classes** (1 raiz + 25 subclasses). Isso permite capturar erros no nível certo: `except JogoError` para tudo, `except InventarioError` só para erros de inventário.

```
class JogoError(Exception): pass # raiz
class InventarioError(JogoError): pass # ramo
class InventarioCheioError(InventarioError): pass # folha
class ItemNaoEncontradoError(InventarioError): pass
class PersistenciaError(JogoError): pass
class SaveNaoEncontradoError(PersistenciaError): pass
class CampanhaError(JogoError): pass
class VenueBloqueadaError(CampanhaError): pass
# ... 26 classes no total (1 raiz + 25 subclasses)
```

Testado em `tests/test_excecoes.py`.

5.6 Bridge — Ponte JS ↔ Domínio Python (bônus)

Categoria GoF: Estrutural · **Arquivo:** `bridge/api.py`

O decorator `@_ponte` separa a **abstração** (API amigável ao JS, com retorno JSON) da **implementação** (domínio Python puro com exceções). O frontend chama `window.pywebview.api.atacar(...)` sem conhecer nada do domínio:

```
def _ponte(fn):
    def wrapper(*args, **kwargs):
        try:
            return {"ok": True, "dados": fn(*args, **kwargs)}
        except JogoError as e:
            return {"ok": False, "erro": str(e)}
    return wrapper
```

Nenhuma exceção cruza a ponte — o frontend recebe sempre `{ok, dados}` ou `{ok: false, erro}`.

Tabela Resumo dos Padrões

#	Padrão	Categoria GoF	Arquivo	Evidência
1	Singleton	Criacional	<code>backend/gerenciador.py</code>	<code>_instancia</code> + <code>__new__</code>
2	Factory Method	Criacional	<code>backend/fabricas.py</code>	<code>MusicoFactory.criar()</code> + <code>registrar()</code>
3	Template Method	Comportamental	<code>backend/musico.py</code> + subclasses	<code>ABC</code> + <code>@abstractmethod</code> <code>atacar()</code>
4	Strategy	Comportamental	<code>backend/moves.py</code> + <code>show.py</code>	<code>MOVES_BASE</code> + move em runtime
5	Hierarquia de Exceções	Estrutural	<code>backend/excecoes.py</code>	<code>JogoError</code> , raiz de 26 classes
6	Bridge	Estrutural	<code>bridge/api.py</code>	<code>@_ponte</code> separa API JS do domínio

6. Herança e Polimorfismo

6.1 Hierarquia Musico → [Guitarrista, Vocalista, Baterista, Baixista]

```

Musico (ABC)
├── Guitarrista
│   └── Baixista ← herda de Guitarrista (compartilha o atributo forca)
├── Vocalista
└── Baterista
  
```

Todos herdam de `Musico` o mecanismo completo de HP, XP, energia, level-up, inventário e equipamento; cada subclasse acrescenta apenas seu atributo específico. Uma decisão menos óbvia: `Baixista` herda de `Guitarrista` (não direto de `Musico`), porque o baixista reaproveita o atributo `forca` da guitarra e só acrescenta `fe` como recurso diferenciador — isso evita duplicar código.

6.2 Hierarquia Item → [Equipavel, Consumivel]

```

Item (base)
├── Equipavel ← ocupa slot, dá bônus de atributo, não é destruído ao usar
└── Consumivel ← efeito único (cura/energia), destruído ao usar
  
```

`Equipavel` sobrescreve `usar(alvo)` para aplicar o bônus de atributo e validar `classes_permitidas`. `Consumivel` sobrescreve `usar(alvo)` para aplicar `cura` ou `energia` e marcar `consumir_ao_usar = True`.

6.3 Polimorfismo em Ação

- **`atacar()` polimórfico:** `Show.acao_musico(musico, move)` chama `musico.atacar()` sem saber o tipo concreto — o resultado é o dano base daquele tipo, multiplicado pelo fator do move.
- **`usar(alvo)` polimórfico:** `Inventario.usar(item_id, alvo)` chama `item.usar(alvo)` sem saber se é `Equipavel` ou `Consumivel` — cada um executa seu efeito específico.

7. Sobrecarga de Operadores

Tornei as classes mais "pythônicas" sobrescrevendo métodos especiais (dunders). O inventário ganhou três deles, sendo dois adicionados por TDD (teste escrito antes da implementação).

7.1 Inventario.__len__

`len(inv)` retorna o número de itens no inventário. Usado na verificação de capacidade e nos testes.

```
def __len__(self) -> int:
    return len(self._itens)
```

7.2 Inventario.__contains__

`"Pedal de Efeito" in inventario` — busca por nome, delegando para `_buscar()`.

```
def __contains__(self, nome_item: str) -> bool:
    """Permite o idioma pythônico: 'Pedal de Efeito' in inventario."""
    return self._buscar(nome_item) is not None
```

Testes em `tests/test_inventario.py` (`test_contains_*`).

7.3 Inventario.__repr__

`repr(inv)` → `"Inventario(3/20 itens)"` — representação legível para debug.

```
def __repr__(self) -> str:
    return f"Inventario({len(self._itens)}/{self.capacidade} itens)"
```

Testes em `tests/test_inventario.py` (`test_repr_*`).

7.4 Musico.__del__

Destrutor — registra no console o fim de vida do objeto ao removê-lo da memória.

```
def __del__(self):
    print(f" [-] Músico '{self._nome}' removido da memória.")
```

Tabela Resumo dos Dunders

Dunder	Classe	Semântica
<code>__len__</code>	<code>Inventario</code>	<code>len(inv)</code> — contagem de itens
<code>__contains__</code>	<code>Inventario</code>	<code>"nome" in inv</code> — busca por nome
<code>__repr__</code>	<code>Inventario</code>	<code>repr(inv)</code> → <code>"Inventario(N/cap itens)"</code>
<code>__del__</code>	<code>Musico</code>	destrutor — ciclo de vida

8. Qualidade e Testes

8.1 Suíte pytest

Métrica	Valor
Testes passando	361
Testes falhando	0
Cobertura (<code>backend/</code>)	95%
Limiar da rubrica	≥ 60% — bem acima

O `backend/main.py` (um script de demonstração via CLI, nunca chamado pelos testes de domínio) é omitido da medição via `.coveragerc`; sem essa omissão a cobertura fica em 86%, ainda muito acima do limiar. Comando para reproduzir:

```
pytest --cov=backend --cov-report=term-missing -q
```

Os 361 testes cobrem: músicos (criação, atributos, level-up, energia), inventário (adicionar/remover/usar, dunders), itens (equipável/consumível), fábricas, combate (Show, golpes, cansaço, crítico), campanha (venues, fama, cache, cooldowns), persistência (round-trip JSON), exceções, a API da bridge e testes de integração.

9. Decisões de Design

As decisões abaixo foram tomadas ao longo do desenvolvimento, em geral depois de testar o jogo na prática:

Decisão	Racional
Backend autoritativo; frontend só renderiza	Separa domínio de apresentação e torna o domínio testável sem UI
<code>Baixista</code> herda de <code>Guitarrista</code>	Compartilha <code>forca</code> ; evita duplicar atributo e métodos de bônus
<code>@_ponte</code> captura toda exceção do domínio	Nenhuma exceção crua cruza para o JS; resposta sempre padronizada
Energia unificada na classe base <code>Musico</code>	Simplifica o modelo — todos os tipos usam o mesmo recurso, sem estados paralelos
Slots de equipamento reversíveis	<code>desequipar()</code> libera o slot; o bônus é recalculado dinamicamente em <code>atacar()</code>
<code>ItemFactory</code> com lambdas como fábricas	Lambdas com <code>**kw</code> permitem customizar atributos (ex.: baú com bônus maior) sem subclasses extras
Loja como ponto fixo do mapa, não na van	Ao jogar percebi que comprar dentro da van confundia; a van só equipa/usa itens já adquiridos

10. Limitações Conhecidas

Por honestidade técnica, registro o que ficou de fora desta entrega:

- **Harnesses de teste do frontend (JS).** Mantive duas páginas-harness para experimentar a batalha e o minigame de ritmo isoladamente no navegador. Ambas têm pequenos bugs de timing/retorno conhecidos. Eles afetam **apenas os harnesses de teste manual** — o jogo real, jogado por `python bridge/app.py`, funciona corretamente, e a suíte automatizada (`pytest`) cobre o backend Python, não esses harnesses. Optei por não investir tempo neles porque não impactam a jogabilidade nem a cobertura.
 - **Executável `.exe` standalone.** O empacotamento via PyInstaller está documentado e funciona na maioria dos casos, mas depende do WebView2 Runtime na máquina destino. O caminho de execução garantido e recomendado continua sendo rodar do código-fonte (ver `docs/empacotamento.md`).
-

11. Conclusão

O **Decibéis** cumpre o objetivo de demonstrar os conceitos centrais de POO em Python 3.12 dentro de um jogo de fato jogável:

- **Herança** em duas hierarquias reais (`Musico` → 4 subclasses; `Item` → 2 subtipos), incluindo a relação não-óbvia `Baixista` → `Guitarrista`.
- **Polimorfismo** em `atacar()` e `usar()` — o código cliente nunca precisa saber o tipo concreto.
- **Encapsulamento** — atributos protegidos (`_`) e privados (`__`) acessados por getters/setters com validação.
- **Abstração** — `Musico` (ABC) define a interface; `Item` define o contrato base.
- **6 padrões GoF** documentados com evidência em código: Singleton, Factory Method, Template Method, Strategy, Hierarquia de Exceções e Bridge.
- **Sobrecarga de operadores** (`__len__`, `__contains__`, `__repr__`, `__del__`).
- **Cobertura de testes** de 95% no `backend/`, com 361 testes verdes.

O jogo roda de ponta a ponta: menu → criar banda → overworld 2D → batalha com minigame de ritmo → level-up → loja → progressão até o chefe final, O Empresário.

Referências

- Gamma, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- Python Software Foundation. *Python 3.12 Data Model — Special method names*. docs.python.org.
- pywebview. *Documentation 6.x*. pywebview.flowrl.com.
- Especificação da Disciplina de POO — UTFPR, 2026.